Part 3 — YARN Capacity Scheduler

This part is the YARN queue setup on the lab cluster. I configured two Capacity Scheduler queues, submitted a WordCount to each queue, and then showed preemption: production needed memory that development was holding, and the ResourceManager killed development containers so production could start.

The cluster has two NodeManagers. Each one advertises 4096 MB and 2 vcores, so the whole pool is 8192 MB and 4 vcores. The scheduler class is CapacityScheduler. The resource calculator is DefaultResourceCalculator, which places containers by memory. I submitted every job from the yarn-client pod. That pod mounts the YARN config. I did not run the client inside the ResourceManager pod, because that pod only has 2 GiB and a second JVM in there gets killed.

capacity-scheduler.xml

This is the file the ResourceManager is actually using. Production is guaranteed 60% of the cluster and is not allowed to grow past 80%. Development is guaranteed 40% and is allowed to grow to 100% when nobody else needs the memory. Preemption is turned on for both queues (disable_preemption is false), and observe_only is false, so the monitor is allowed to kill containers rather than only logging what it would do.

user-limit-factor is the extra setting that made a single lab user able to reach those maximums. The default factor is 1, which means one user can only hold the queue's guaranteed share. Development's guarantee is 40%, so one user would have stopped around there and could never fill the cluster. I set development to 2.5 (40% times 2.5 is 100%) and production to 2 (60% times 2 is more than its 80% cap, so the maximum-capacity still stops it at 80%).

```
<?xml version="1.0"?>
<configuration>
  <property>
    <name>yarn.scheduler.capacity.maximum-applications</name>
    <value>10000</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.maximum-am-resource-percent</name>
    <value>0.5</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.resource-calculator</name>
    <value>org.apache.hadoop.yarn.util.resource.DefaultResourceCalculator</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.queues</name>
    <value>production,development</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.production.capacity</name>
    <value>60</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.production.maximum-capacity</name>
    <value>80</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.production.disable_preemption</name>
    <value>false</value>
  </property>
  <property>
```

```
    <name>yarn.scheduler.capacity.root.production.user-limit-factor</name>
    <value>2</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.development.capacity</name>
    <value>40</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.development.maximum-capacity</name>
    <value>100</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.development.disable_preemption</name>
    <value>false</value>
  </property>
  <property>
    <name>yarn.scheduler.capacity.root.development.user-limit-factor</name>
    <value>2.5</value>
  </property>
  <property>
    <name>yarn.resourcemanager.monitor.capacity.preemption.observe_only</name>
    <value>false</value>
  </property>
  <property>
    <name>yarn.resourcemanager.scheduler.monitor.enable</name>
    <value>true</value>
  </property>
  <property>
    <name>yarn.resourcemanager.scheduler.monitor.policies</name>

<value>org.apache.hadoop.yarn.server.resourcemanager.monitor.capacity.ProportionalCapacityPreemptionPolicy</val
ue>
  </property>
</configuration>
```

On the scheduler page the two bars match that file. Production's configured capacity is 60% and its max is 80%. Development's configured capacity is 40% and its max is 100%. Both WordCount jobs are already finished in this shot, which is why Used Capacity is back at 0%. The queue limits are what I am showing here.

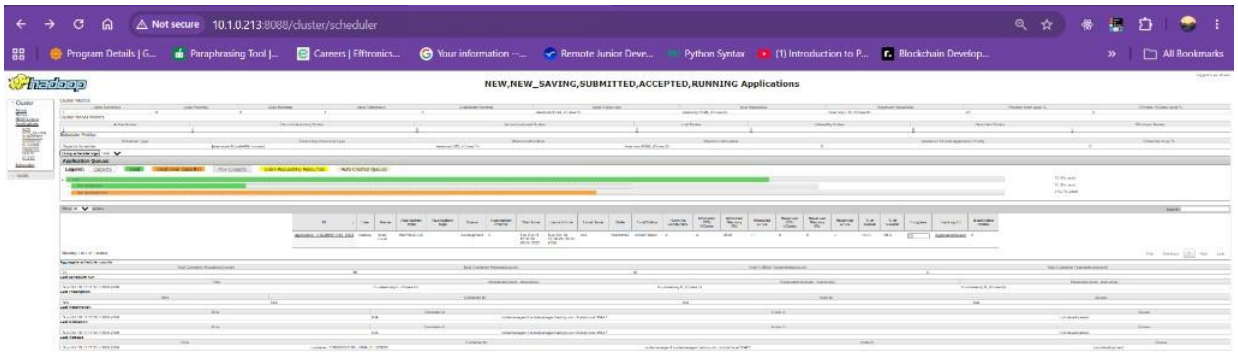


Figure 1. ResourceManager scheduler. production capacity 60% (max 80%), development capacity 40% (max 100%). WordCount application_1792266501185_0002 is on production and application_1792266501185_0004 is on development. Both are FINISHED.

WordCount on both queues

I put the input in HDFS first, from the NameNode, because the NodeManager's default filesystem is the local disk and a put from that pod does not land in HDFS. The input is /user/root/wordcount-input: 8 files,

5,000,000 bytes each, replication 2. I set the split size to 1 MB so the job had enough map tasks to stay on the cluster while I submitted the second one.

Production job. The queue name is a job configuration property, and the framework has to be yarn. If the framework is left as local, the job runs inside the client and never shows up on the ResourceManager.

```
kubectl exec -n hadoop yarn-client -- bash -lc '
hadoop jar /opt/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar wordcount \
  -Dmapreduce.framework.name=yarn \
  -Dmapreduce.job.queuename=production \
  -Dmapreduce.input.fileinputformat.split.maxsize=1048576 \
  /user/root/wordcount-input \
  /user/root/wordcount-output-prod
'
```

That became application_1792266501185_0002. It finished SUCCEEDED on the production queue. The tracking page shows the final state, the queue, and the elapsed time.

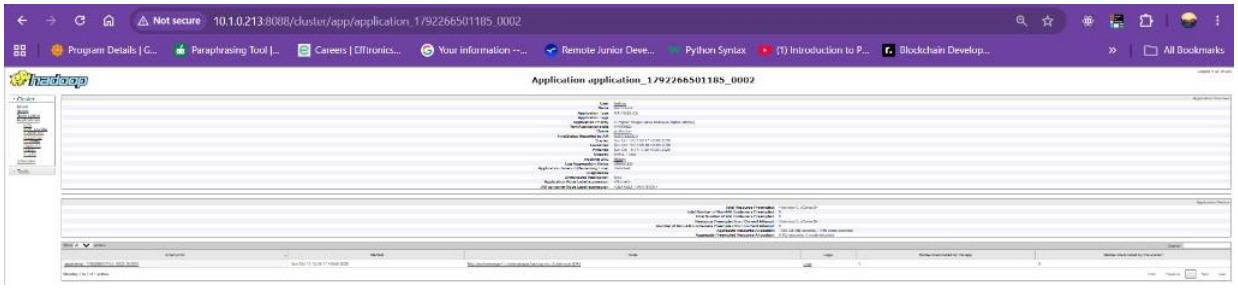


Figure 2. application_1792266501185_0002, WordCount, queue production, FINISHED / SUCCEEDED.

I submitted the development job the same way, against the same input, while the production job was the one I wanted overlapping it. Output goes to a different directory so the two jobs do not fight over the same path.

```
kubectl exec -n hadoop yarn-client -- bash -lc '
hadoop jar /opt/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar wordcount \
  -Dmapreduce.framework.name=yarn \
  -Dmapreduce.job.queuename=development \
  -Dmapreduce.input.fileinputformat.split.maxsize=1048576 \
  /user/root/wordcount-input \
  /user/root/wordcount-output-dev
'
```

That became application_1792266501185_0004. It also finished SUCCEEDED, on the development queue. The ApplicationSummary for this job was memorySeconds=1395958, vcoreSeconds=1150, totalAllocatedContainers=42, and preemptedMemorySeconds=0. The production WordCount summary was the same story: it succeeded, and nothing was preempted.

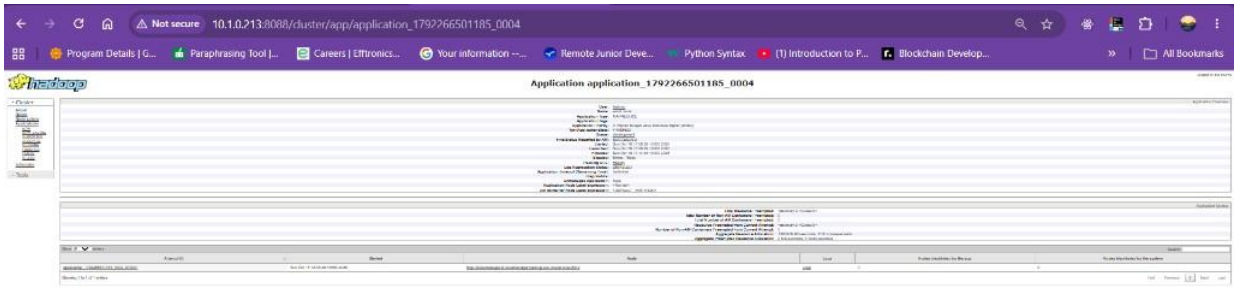


Figure 3. application_1792266501185_0004, WordCount, queue development, FINISHED / SUCCEEDED.

Why WordCount did not trigger preemption

Both WordCounts are valid queue submissions, and they are the jobs I am keeping for the two-queue part. They are not the preemption proof. A map task in that job asks for the default 1024 MB. With 8192 MB in the pool, the ResourceManager could place the second job's containers in memory that was still free. Preemption only runs when a queue is under its guarantee, another queue is over its guarantee, and the starved queue's next container does not fit in the free memory. Free memory was left, so both jobs finished with preemptedMemorySeconds=0.

Filling the cluster so production has to reclaim

SleepJob is not inside hadoop-mapreduce-examples. It lives in the MapReduce jobclient tests jar, and that jar's main class is a test driver that wants JUnit on the classpath. The invocation that works is hadoop org.apache.hadoop.mapreduce.SleepJob with the tests jar, junit, and hamcrest on HADOOP_CLASSPATH.

I sized every container at 2048 MB, including the ApplicationMaster. Two of those fill one NodeManager (4096 MB). Four of them fill the cluster (8192 MB) and leave nothing for a new 2048 MB container. The maps sleep for 10 minutes, so they are still holding the memory when I submit production. Eight maps are requested so the job keeps asking for containers after the first four are running.

```
kubectl exec -n hadoop yarn-client -- bash -lc '
export HADOOP_CLASSPATH=/opt/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-client-jobclient-3.3.6-
tests.jar:/opt/hadoop/share/hadoop/tools/lib/junit-4.13.2.jar:/opt/hadoop/share/hadoop/tools/lib/hamcrest-core-
1.3.jar
hadoop org.apache.hadoop.mapreduce.SleepJob \
  -Dmapreduce.framework.name=yarn \
  -Dmapreduce.job.queuename=development \
  -Dyarn.app.mapreduce.am.resource.mb=2048 \
  -Dyarn.app.mapreduce.am.resource.cpu-vcores=1 \
  -Dmapreduce.map.memory.mb=2048 \
  -Dmapreduce.map.cpu.vcores=1 \
  -m 8 -r 0 -mt 600000 -rt 1000
'
```

I waited until both nodes reported 0 MB free, then submitted production. Production asks for the same 2048 MB container size, so its ApplicationMaster cannot be placed until something is taken back.

```
kubectl exec -n hadoop yarn-client -- bash -lc '
export HADOOP_CLASSPATH=/opt/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-client-jobclient-3.3.6-
tests.jar:/opt/hadoop/share/hadoop/tools/lib/junit-4.13.2.jar:/opt/hadoop/share/hadoop/tools/lib/hamcrest-core-
1.3.jar
hadoop org.apache.hadoop.mapreduce.SleepJob \
```

```
  -Dmapreduce.framework.name=yarn \
  -Dmapreduce.job.queuename=production \
  -Dyarn.app.mapreduce.am.resource.mb=2048 \
  -Dyarn.app.mapreduce.am.resource.cpu-vcores=1 \
  -Dmapreduce.map.memory.mb=2048 \
  -Dmapreduce.map.cpu.vcores=1 \
  -m 4 -r 0 -mt 180000 -rt 1000
'
```

Development is application_1792313485971_0001. Production is application_1792313485971_0002. On the scheduler page both are on the cluster at the same time: development is the one that filled the nodes, and production is the queue that is under its 60% guarantee and waiting on memory.

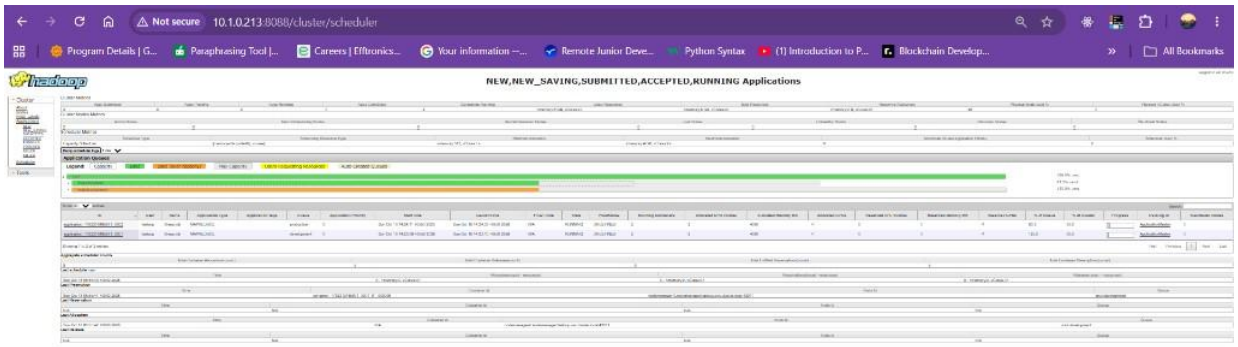


Figure 4. Scheduler while the sleep jobs overlap. application_1792313485971_0001 is on development and application_1792313485971_0002 is on production.

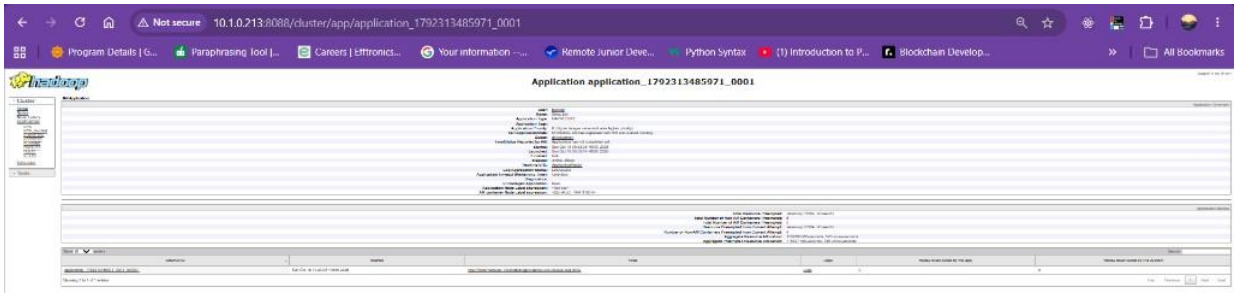


Figure 5. application_1792313485971_0001, Sleep job, queue development. This is the application the scheduler took containers from.

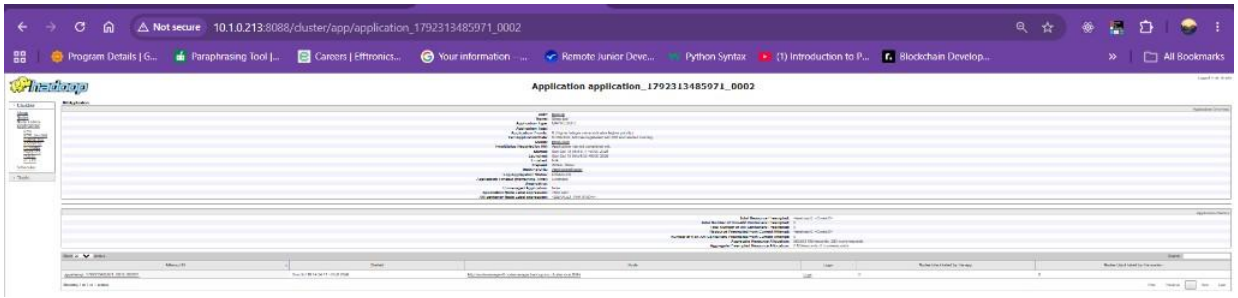


Figure 6. application_1792313485971_0002, Sleep job, queue production. This is the application that received the reclaimed memory.

How preemption worked

The monitor is ProportionalCapacityPreemptionPolicy. yarn.resourcemanager.scheduler.monitor.enable is true, and the monitor checks every 3000 ms. On each pass it looks at what each queue is guaranteed versus what it is actually using.

Development's guarantee is 40% of 8192 MB, about 3277 MB. The sleep job was using the whole 8192 MB, which is allowed by its maximum of 100%, but it is well above the guarantee. Production's guarantee is 60%, about 4915 MB, and it was sitting at 0 MB with a 2048 MB ApplicationMaster that could not be placed. That is the condition the policy is built for.

It does not kill a container on the first sight of that imbalance. max_wait_before_kill is 15000 ms, so production has to stay starved for 15 seconds. After that the policy chooses containers from the queue that is over its share. It prefers task containers over the ApplicationMaster, which is why the log says Non-AM container preempted. The container it killed was 2048 MB and 1 vcore, which is exactly one map slot. That memory went back to the pool and production's ApplicationMaster was placed.

Two numbers in yarn-site.xml decide whether a 2048 MB container is actually eligible. total_preemption_per_round is 0.8, so one pass is allowed to reclaim up to 80% of the cluster. A 2048 MB container is 25% of 8192 MB, so it fits in that budget. natural_termination_factor is 1. That factor is a multiplier on the amount the policy tries to take in one pass. At the default of 0.2 it only asked for 20% of the excess, and 20% of the excess was smaller than one 2048 MB container, so it selected nothing and the preempted counter stayed at 0. At 1 it asks for the full excess, which is large enough to cover one container. max_ignored_over_capacity is 0, so development does not get a free band above its guarantee. select_based_on_reserved_containers is true, so a production request that is reserved but not running still counts as demand.

These are the yarn-site.xml values the ResourceManager loaded:

```
yarn.resourcemanager.scheduler.class
  = org.apache.hadoop.yarn.server.resourcemanager.scheduler.capacity.CapacityScheduler
yarn.scheduler.minimum-allocation-mb = 512
yarn.scheduler.maximum-allocation-mb = 4096
yarn.scheduler.maximum-allocation-vcores = 2
yarn.resourcemanager.scheduler.monitor.enable = true
yarn.resourcemanager.monitor.capacity.preemption.monitoring_interval = 3000
yarn.resourcemanager.monitor.capacity.preemption.max_wait_before_kill = 15000
yarn.resourcemanager.monitor.capacity.preemption.total_preemption_per_round = 0.8
yarn.resourcemanager.monitor.capacity.preemption.natural_termination_factor = 1
yarn.resourcemanager.monitor.capacity.preemption.max_ignored_over_capacity = 0
yarn.resourcemanager.monitor.capacity.preemption.select_based_on_reserved_containers = true
```

The same thing kept happening after the first container. Production still wanted more 2048 MB maps, development still had maps asleep, and the monitor killed the next development container. That is why the log has a run of lines rather than one.

CLI proof

yarn application -status on the development job is the summary. The line that shows the reclaim is Aggregate Resource Preempted. In this shot it is 1059501 MB-seconds and 509 vcore-seconds. Aggregate Resource Allocation above it is everything the job was given, including what was later taken back. The queue is

development and the state is still RUNNING, which is what I want in the screenshot: the job is alive and the preempted counter is already non-zero.

```
kubectl exec -n hadoop yarn-client -- bash -lc 'export HADOOP_CLIENT_OPTS=-Xmx128m; yarn application -status
application_1792313485971_0001'
```

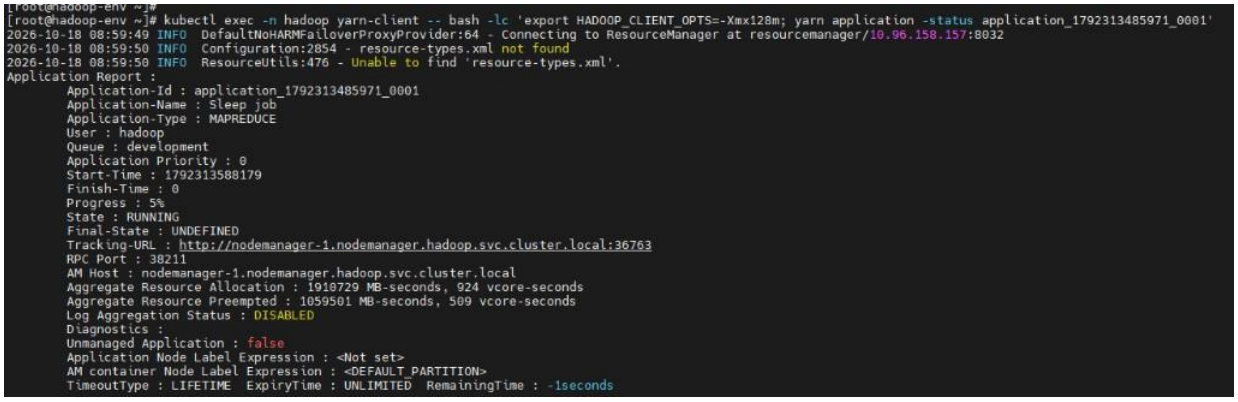


Figure 7. yarn application -status application_1792313485971_0001. Queue development. Aggregate Resource Preempted is 1059501 MB-seconds, 509 vcore-seconds.

The ResourceManager log names the containers. The first one I recorded was at 08:54:32, container_1792313485971_0001_01_000004, resource=<memory:2048, vCores:1>. The lines continue through container_..._000019 at 08:59:14. Every one of them is a non-AM container from appattempt_1792313485971_0001_000001, the development attempt, and every one is 2048 MB and 1 vcore.

```
kubectl logs -n hadoop deploy/resourcemanager --since=30m | grep "container preempted"
```

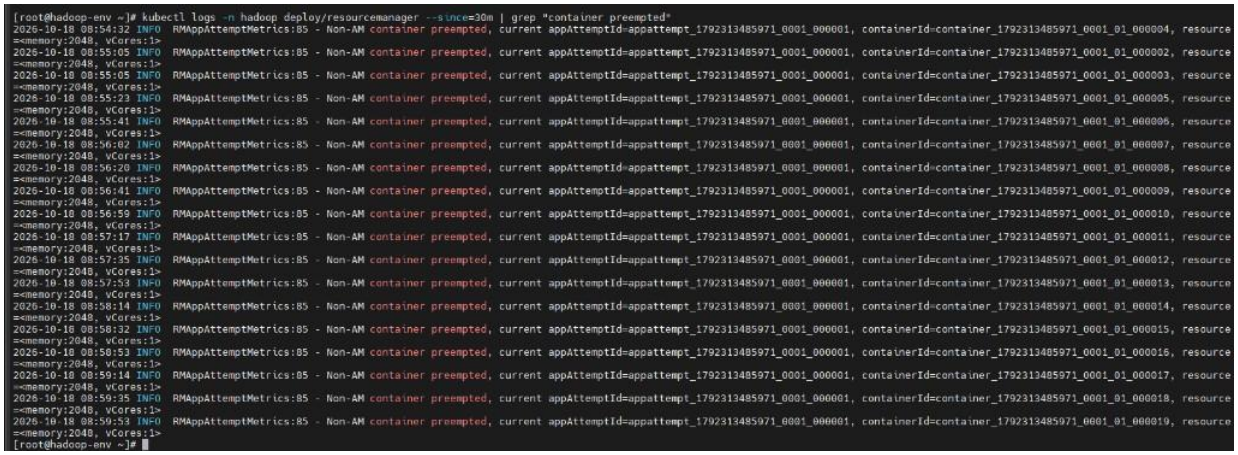


Figure 8. ResourceManager log. Non-AM containers preempted from the development attempt, each 2048 MB and 1 vcore, from 08:54:32 through 08:59:14.

That is the preemption result. Development was allowed to borrow the whole cluster because its maximum is 100%. When production, which is guaranteed 60%, could not place a 2048 MB container, the Capacity Scheduler waited 15 seconds and then killed development map containers until production could run. The

WordCount pair shows the two queues accepting work. The sleep pair, the preempted counter, and the container-preempted log show the reclaim.